Run AI Locally with Ollama & Python

Your Complete Beginner's Reference Guide

No cloud. No cost. No prior experience needed.

Watch YouTube Video here: https://youtu.be/Fp8GWFzvcqs

---

1. What is Ollama?

Ollama is a free, open-source tool that lets you download and run powerful AI language models directly on your own computer — no internet required, no API keys, no monthly bills.

> **Think of it this way:**
> Ollama is like an "app store + launcher" for AI models. You pick a model, Ollama installs it, and you start chatting — all offline, all private, all free.

Why use Ollama?

- Privacy — your prompts never leave your machine
- Zero cost — no API billing, no subscriptions
- Speed — no network latency, runs on your local hardware
- Full control — choose any model, switch anytime
- Works offline — no internet needed once models are downloaded

---

2. Installing Ollama

Step 1: Download

Visit the official Ollama website and download the installer for your operating system:
`https://ollama.com`

Step 2: Install

- Windows: Run the downloaded .exe installer and follow the prompts
- macOS: Open the .dmg file and drag Ollama to your Applications folder
- Linux: Run the one-line install script from the website

Step 3: Verify

Open your terminal (Command Prompt, PowerShell, or Terminal) and run:

```
ollama --version
```

You should see a version number. Ollama also appears in your system tray once running.

> **Quick Check:**
> Open your browser and go to http://localhost:11434 — if Ollama is running, you will see a confirmation message. This is the local server your Python code will talk to.

---

3. Essential Ollama Commands

All commands are run in your terminal (Command Prompt / PowerShell / Terminal).

Core Commands

Command	What it does
`ollama run <model>`	Downloads (if needed) and starts an interactive chat session with the model
`ollama pull <model>`	Downloads a model without launching it immediately
`ollama list`	Shows all models currently downloaded on your machine
`ollama rm <model>`	Deletes a downloaded model to free up disk space
`ollama ps`	Shows which models are currently loaded in memory
`ollama show <model>`	Displays detailed info about a model (parameters, license, etc.)
`ollama stop <model>`	Unloads a running model from memory immediately
`ollama help`	Lists all available commands and usage flags

Interactive Mode Shortcuts (inside ollama run)

Command	What it does
`/bye`	Exit the chat session and return to the terminal
`/show info`	View details about the currently running model
`/set parameter temperature 0.7`	Temporarily adjust the model's temperature setting
`/set parameter num_ctx 4096`	Change the context window length for longer conversations

Recommended Models for Beginners

Command	What it does
`ollama run llama3`	Meta's Llama 3 — excellent all-around performance
`ollama run gemma2:2b`	Google's Gemma 2B — very fast, great for low-end hardware
`ollama run mistral`	Mistral 7B — strong reasoning, good for code and writing
`ollama run phi3`	Microsoft's Phi-3 — tiny but surprisingly capable

4. Python Code — Full Walkthrough

Prerequisites

1. Python installed — download from https://python.org
2. Ollama running in the background (check your system tray)
3. A model downloaded — e.g., run: ollama pull gemma2:2b
4. The requests library — install with: pip install requests

The Complete Python Script

Download the code here:
https://github.com/thetechblackboard/Ollama_LLM_Course/blob/main/Simple-Ollama-call.py

```
import requests
import json

def chat_with_llm(prompt, model="gemma2:2b"):
    url = "http://localhost:11434/api/generate"

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False
    }

    response = requests.post(url, json=payload)
    result = response.json()
    return result["response"]

answer = chat_with_llm("How is an AI agent different from Agentic AI?")
print(answer)
```

Line-by-Line Explanation

Imports

```
import requests
import json
```

- requests — a Python library for making HTTP calls (sending data to a web address)
- json — a library for handling JSON-formatted data (the format APIs use)
- These lines run immediately when the script starts

Defining the Function

```
def chat_with_llm(prompt, model="gemma2:2b"):
```

- def — the keyword that tells Python you are defining a function
- chat_with_llm — the name you give the function (you choose this)
- prompt — the question or message you want to send to the AI
- model="gemma2:2b" — a default value; if you don't specify a model, it uses gemma2:2b
- The function is only registered in memory here — it does NOT run yet

The URL

```
url = "http://localhost:11434/api/generate"
```

- localhost — means your own computer (not the internet)
- 11434 — the port number Ollama listens on
- /api/generate — the endpoint that generates text responses

The Payload

```
payload = {
    "model": model,
    "prompt": prompt,
    "stream": False
}
```

- payload — a Python dictionary (key-value pairs) containing the request data
- model — which AI model to use
- prompt — the question you're asking
- stream: False — tells Ollama to return the complete response all at once (not word by word)

Sending the Request

```
response = requests.post(url, json=payload)
```

- requests.post() — sends an HTTP POST request (like submitting a form)
- url — where to send the request (your local Ollama server)
- json=payload — automatically converts the Python dictionary to JSON format
- This line WAITS until the AI responds before moving on

Getting the Answer

```
result = response.json()
return result["response"]
```

- response.json() — converts the raw response into a Python dictionary
- result["response"] — extracts just the AI's text answer from the dictionary

Calling the Function and Printing

```
answer = chat_with_llm("How is an AI agent different from Agentic AI?")
print(answer)
```

- This is where the function actually runs for the first time
- The return value (the AI's answer) is stored in the variable answer
- print(answer) displays the result in the terminal

---

5. Execution Flow — How the Program Runs

Also Check:
https://github.com/thetechblackboard/Ollama_LLM_Course/blob/main/Execution_Steps_Simple_Ollama-call.md

When you run the script, Python executes it top to bottom. Here is the exact sequence:

1. Import libraries (requests and json are loaded into memory)
2. Define function (chat_with_llm is registered — NOT run yet)
3. Call the function with your prompt
4. Inside the function: the URL is set
5. Inside the function: the payload dictionary is built
6. HTTP POST request is sent to the Ollama server
7. Ollama receives the prompt and runs the AI model
8. Ollama sends the response back as JSON
9. response.json() parses the JSON into a Python dictionary
10. The answer is extracted and returned
11. The answer is stored in the variable "answer"
12. print(answer) displays the result on your screen

---

6. Homework — Add Error Handling

The code above works, but what if the server is down or something goes wrong? Here is the improved version with error handling. Try to understand and run it yourself:

```
import requests
```

```
def chat_with_llm(prompt, model="gemma2:2b"):
    url = "http://localhost:11434/api/generate"

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False
    }

    response = requests.post(url, json=payload)

    if response.status_code == 200:
        result = response.json()
        return result.get("response", "No response key found")
    else:
        return f"Error: {response.status_code}"

answer = chat_with_llm("Explain machine learning in simple terms")
print(answer)
```

What's New in This Version

- response.status_code == 200 — 200 means success in HTTP (like a green light)
- result.get("response", ...) — safely gets the key; returns a fallback message if it doesn't exist
- The else block — handles errors gracefully instead of crashing the program

Challenge:
Try changing the prompt to something fun — 'Explain AI to a 5-year-old', 'Write me a haiku about Python', or 'What is the meaning of life?' Post your results in the comments!

---

7. What's Coming Next in This Series

This is just the beginning! Here is what the upcoming videos will cover:

Part 2 — Adding Personas to Your AI

- Give your AI a personality using system prompts
- Use the chat-based API with roles: system, user, and assistant
- Turn it into a teacher, a coding mentor, or even a sarcastic assistant
- Understand how system prompts shape the AI's behavior

Part 3 — Building a Simple Chatbot

- Add conversation memory using a growing messages list
- Build a realistic, multi-turn chatbot that remembers context
- Understand how context windows work in LLMs

8. Quick Reference Cheat Sheet

Common Errors and Fixes

Command	What it does
`Connection refused`	Ollama is not running — check your system tray and restart it
`Model not found`	Run: ollama pull <model-name> to download it first
`ModuleNotFoundError: requests`	Run: pip install requests in your terminal
`Slow response`	Normal for large models on CPU — try gemma2:2b for speed
`Out of memory`	Try a smaller model like phi3 or gemma2:2b

Key Concepts Glossary

Command	What it does
`LLM`	Large Language Model — the AI brain (e.g., Llama 3, Gemma, Mistral)
`Ollama`	The tool that runs LLMs locally on your computer
`API`	Application Programming Interface — a way for programs to talk to each other
`HTTP POST`	A method for sending data to a web address (like submitting a form)
`JSON`	JavaScript Object Notation — a common data format used by APIs
`localhost`	Refers to your own computer (127.0.0.1)
`Port 11434`	The network port Ollama uses to communicate
`stream: False`	Return the full response at once, not word by word
`status_code 200`	HTTP code meaning the request was successful
`Payload`	The data package you send to the API

Happy learning! Drop your questions in the comments and subscribe for the next parts in the series.

Remember: The best way to learn is to run the code, break it, and fix it yourself.